QUIZ Pertemuan 5 — Polymorphism dan Duck Typing pada Python

Nama	**:**	**Jihan Maharani**
NIM	**:**	**250660221031**
Kelas	**:**	**Sistem Informasi 3-A**
Mata Kuliah	**:**	**Pemrograman Berorientasi Objek**

SOAL

1. **Soal 1 — Konsep Polymorphism**
   Jelaskan pengertian polymorphism dalam OOP dan berikan satu contoh method yang menghasilkan perilaku berbeda pada class yang berbeda.

   Jawaban:
   Polymorphism adalah salah satu konsep OOP yang memungkinkan method dengan nama yang sama memiliki perilaku atau hasil yang berbeda tergantung pada object yang menggunakannya.
   Contoh:

```
class Mahasiswa:
    def tampilkan_peran(self):
        return "Mahasiswa"

class Dosen:
    def tampilkan_peran(self):
        return "Dosen"
```

   Kedua class memiliki method tampilkan_peran(), tetapi menghasilkan nilai yang berbeda sesuai dengan class masing-masing.

2. **Soal 2 — Overriding dan Polymorphism**
   Jelaskan hubungan antara overriding dan polymorphism. Mengapa overriding menjadi dasar perilaku polimorfik pada Python?

   Jawaban:
   Overriding adalah proses ketika subclass membuat ulang method dari superclass dengan implementasi yang berbeda. Sementara itu, polymorphism memungkinkan method yang sama menghasilkan perilaku berbeda berdasarkan object yang memanggilnya.
   Overriding menjadi salah satu dasar polymorphism pada Python karena subclass dapat memberikan perilaku khusus terhadap method yang diwarisinya. Dengan demikian, pemanggilan method yang sama dapat menghasilkan output berbeda.

3. Soal 3 — Prediksi Output

```
class Pengguna:
    def tampilkan_peran(self):
        return "Pengguna sistem"

class Mahasiswa(Pengguna):
    def tampilkan_peran(self):
        return "Mahasiswa"

class Dosen(Pengguna):
    def tampilkan_peran(self):
        return "Dosen"

for data in [Mahasiswa(), Dosen()]:
    print(data.tampilkan_peran())
```

Tuliskan output dari program tersebut dan jelaskan konsep OOP yang ditunjukkan.

Jawaban:
Output:
Mahasiswa
Dosen

Penjelasan:
Program melakukan perulangan terhadap dua object, yaitu Mahasiswa() dan Dosen(). Keduanya memiliki method tampilkan_peran() dengan hasil yang berbeda karena masing-masing melakukan overriding terhadap method dari superclass Pengguna.
Konsep OOP yang ditunjukkan adalah polymorphism melalui method overriding.

4. Soal 4 — Duck Typing

Fungsi tampilkan() dapat menerima kedua object karena Python menerapkan konsep Duck Typing.
Duck Typing adalah konsep yang memungkinkan suatu object digunakan berdasarkan method atau perilaku yang dimilikinya, bukan berdasarkan hubungan inheritance atau tipe class-nya.
Pada contoh tersebut, class Mahasiswa dan Dosen tidak memiliki hubungan inheritance, tetapi keduanya mempunyai method ringkasan(). Oleh karena itu, keduanya dapat diterima oleh fungsi tampilkan().

5. Soal 5 — Merancang Fungsi Polimorfik

Tuliskan fungsi tampilkan_semua(daftar) yang menerima daftar object beragam dan memanggil method ringkasan() pada setiap object tanpa memeriksa tipe. Jelaskan apa yang terjadi jika sebuah object tidak memiliki method ringkasan().

Jawaban:
Berikut fungsi tampilkan_semua(daftar):

```
def tampilkan_semua(daftar):
   for data in daftar:
      print(data.ringkasan())
```

Contoh penerapan:
class Mahasiswa:

```
    def ringkasan(self):
        return "Mahasiswa — Budi"

class Dosen:
    def ringkasan(self):
        return "Dosen — Sari"

def tampilkan_semua(daftar):
    for data in daftar:
        print(data.ringkasan())

tampilkan_semua([Mahasiswa(), Dosen()])
```

Output:
Mahasiswa — Budi
Dosen — Sari

Penjelasan:
Fungsi tampilkan_semua() melakukan perulangan pada setiap object dan memanggil method ringkasan() tanpa memeriksa tipe object terlebih dahulu.
Jika terdapat object yang tidak memiliki method ringkasan(), Python akan menghasilkan AttributeError karena method tersebut tidak ditemukan pada object.
Jadi, fungsi ini menerapkan konsep polymorphism dan duck typing, yaitu menggunakan object berdasarkan kemampuan atau method yang dimilikinya.